



# 抖音包体积优化体系建设

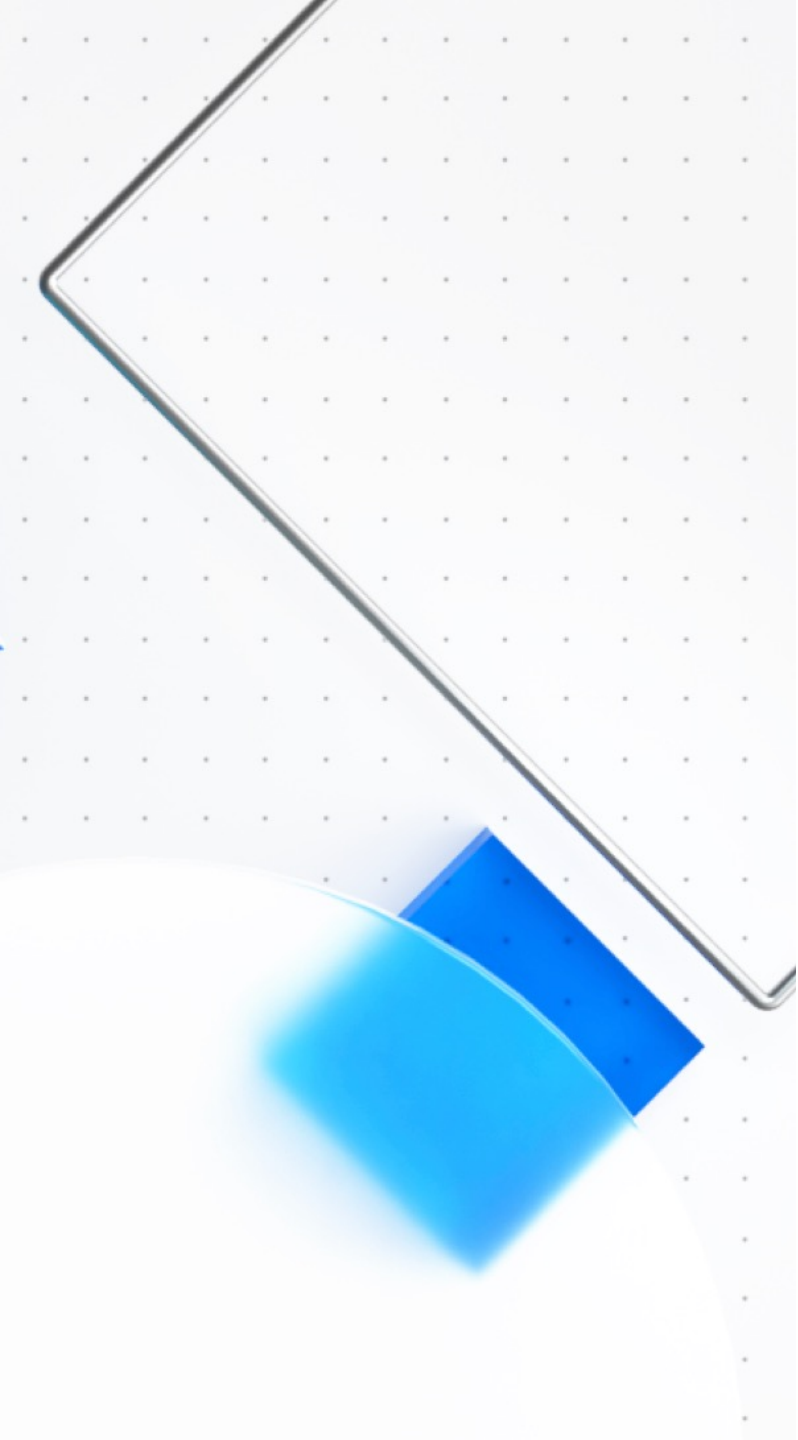
魏馥檀

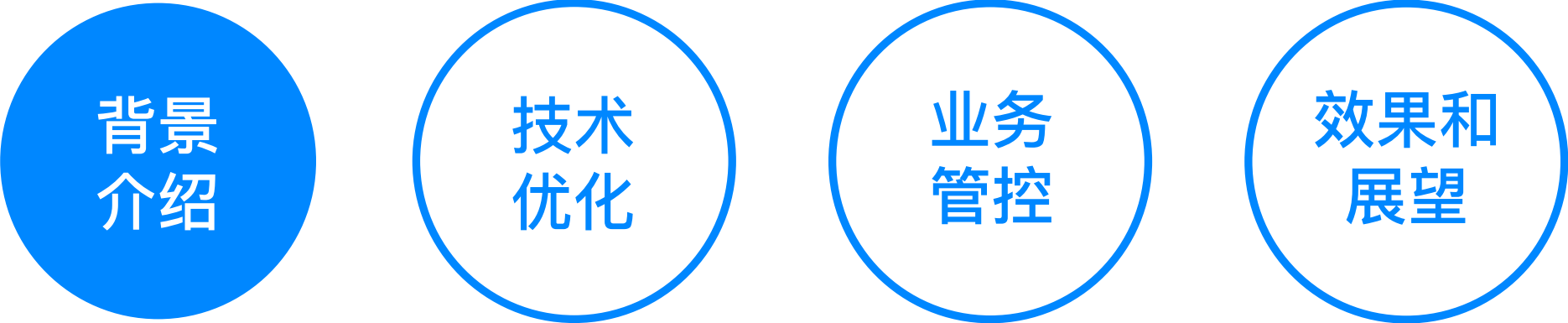
# 讲师自我介绍



## 魏馥檀

2018年本科毕业于南开大学，2019年加入字节跳动抖音Android客户端基础技术团队，长期从事于抖音/抖音极速版包大小相关工作，目前专注于抖音包体积技术优化、包体积平台的开发和技术演进。对Android包体积优化有着较为丰富的经验。

- 
- 01 背景介绍
  - 02 技术优化
  - 03 业务优化
  - 04 效果和展望



背景  
介绍

技术  
优化

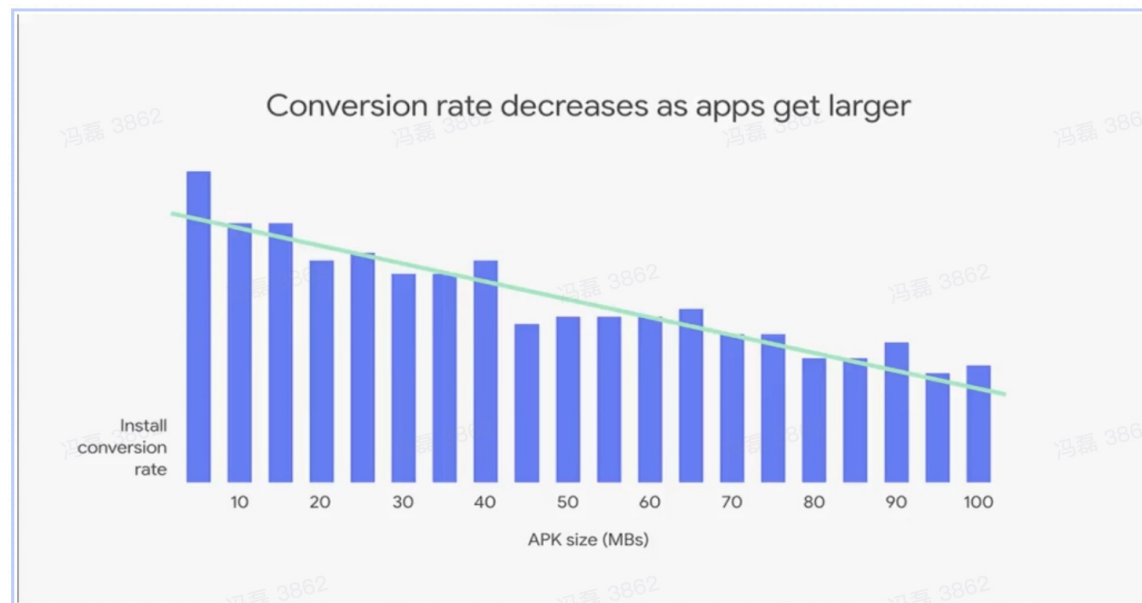
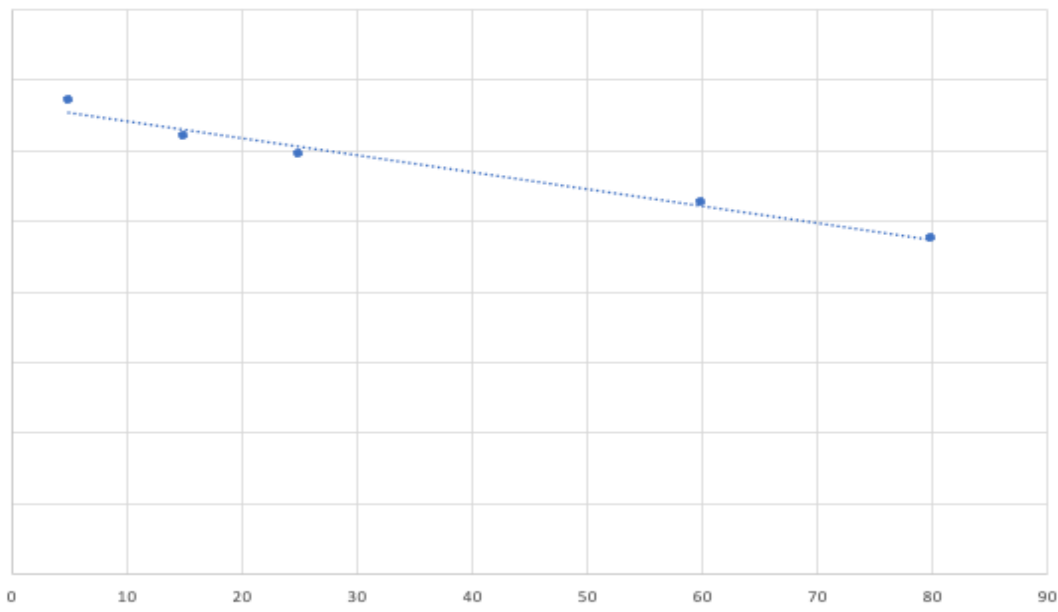
业务  
管控

效果和  
展望

## 01\_包体积影响

抖音包体积实验：

- 在大包80M体积情况下，每M的体积增大会减少约0.3%~0.5%的激活量
- 在小包5~15M体积情况下，每M的体积增大会减少约0.7%的激活量



Google包体积数据：

- 整体呈现包体积增大转化率下降的趋势
- 平均每增加1M，激活率下降0.17%，在低端设备较多的区域，该值还会上升

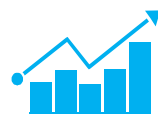
# 01\_包体积优化阶段

优化前期



技术优化多，业务优化偏少

优化中期



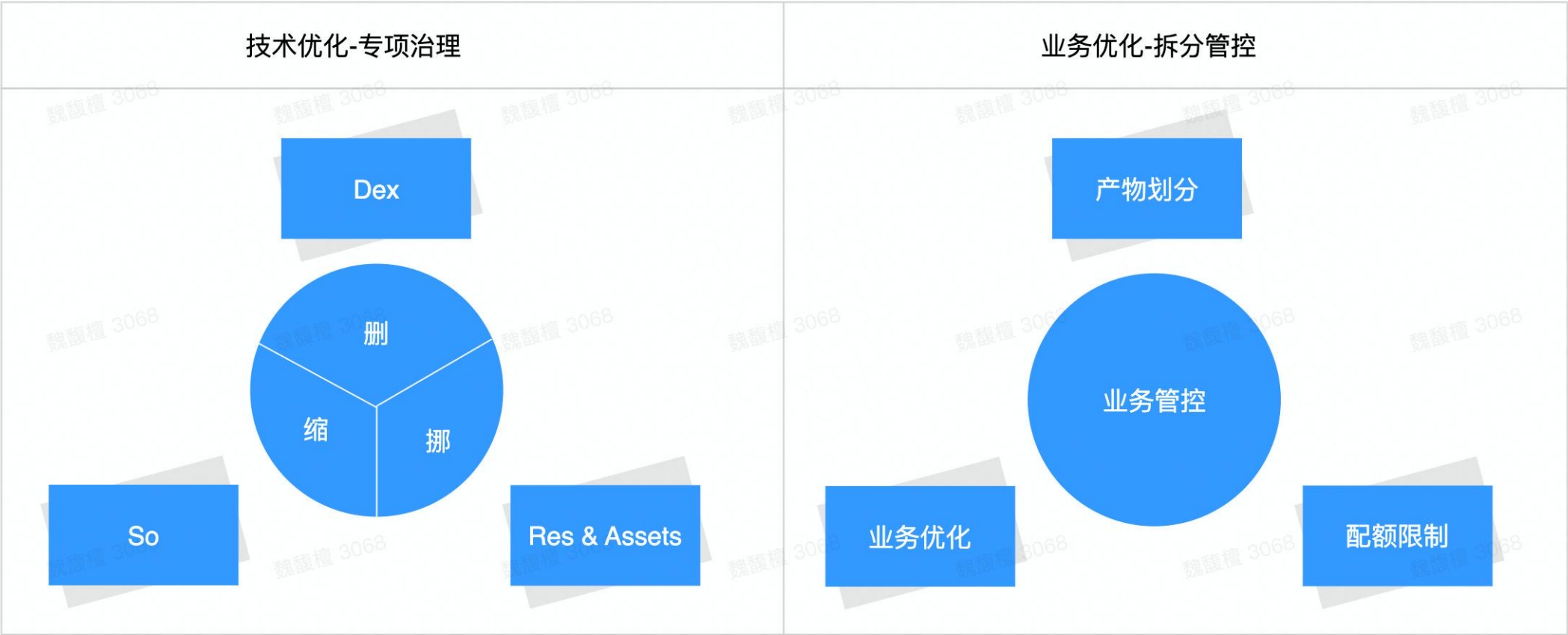
业务劣化加剧，技术优化为主，辅以业务优化

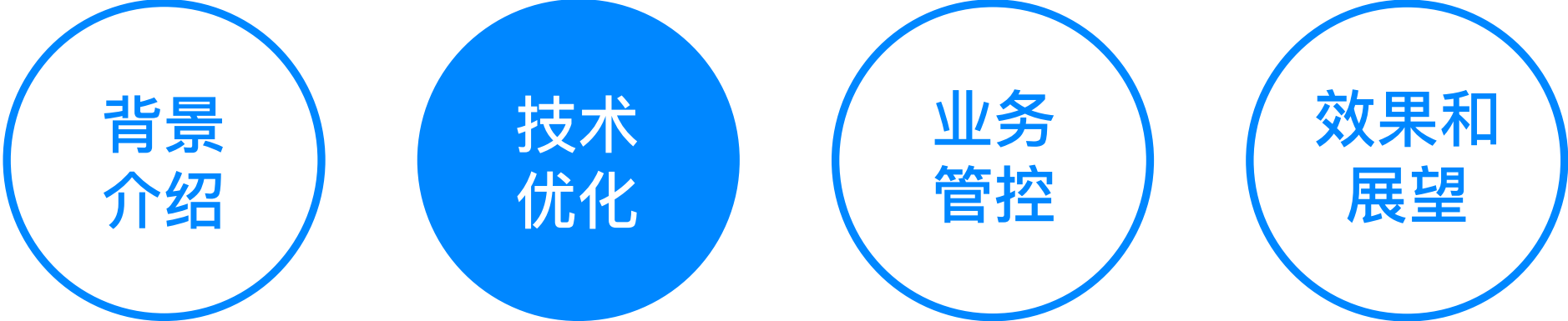
优化后期



业务优化为主，辅以技术优化，包大小随业务迭代变化

# 01\_包体积优化思路





背景  
介绍

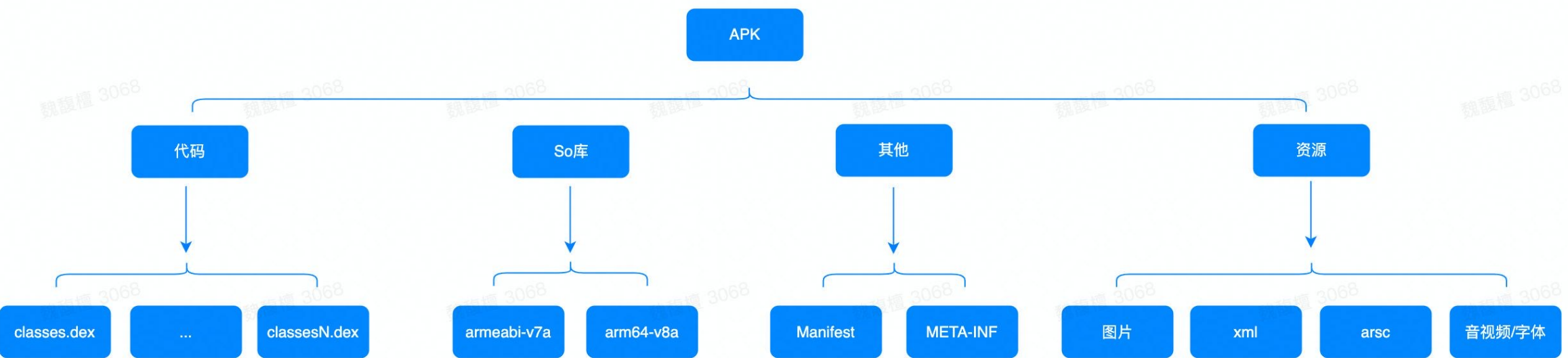
技术  
优化

业务  
管控

效果和  
展望



# 02\_ APK组成分析



## 02\_优化方法论



删：裁剪无用



缩：变换形式；压缩冗余



挪：离线下发；插件化

# 02\_代码优化

|                  | 删                                                   | 缩                                                           |
|------------------|-----------------------------------------------------|-------------------------------------------------------------|
| java => .class   | 删除无副作用代码<br>ProGuard配置优化                            | 常量内联    R文件内联    短方法内联<br>PB字节码优化                           |
| kotlin => .class | Metadata注解优化    Metadata配置文件优化<br>Intrinsics判空字符串优化 | Intrinsics判空指令合并<br>Kotlin基础函数优化    Kotlin语法糖优化             |
| dex              | 跨dex引用缩减    寄存器分配优化<br>SourceFile去除    系统注解删除       | DebugInfo精简    StringBuilder外联<br>常量传播    冗余类型检查去除<br>方法重定义 |

- 基于Java字节码的深度优化
- 适配Kotlin语法糖，精简代码生成
- 基于Dalvik字节码高效精简
- 深度压缩Dalvik可执行文件格式

## 02\_Kotlin 基础函数优化 — npe语句合并

Kotlin class 文件：

```
data class Son(val name: String, val teacher: String, var age: Int = 0)
```

优化点：合并连续的Intrinsics判空语句

```
public Son(@NotNull String name, @NotNull String teacher, int age) {  
    Intrinsics.checkNotNullParameter(name, "name");  
    Intrinsics.checkNotNullParameter(teacher, "teacher");  
    this.name = name;  
    this.teacher = teacher;  
    this.age = age;  
}
```



```
public Son(String name, String teacher, int age) {  
    CheckNpe.throwNpe2(name, teacher);  
    this.name = name;  
    this.teacher = teacher;  
    this.age = age;  
}
```

```
public class CheckNpe {  
    public static void throwNpe1(Object arg1) {  
        if (arg1 == null) {  
            throw new NullPointerException();  
        }  
    }  
    public static void throwNpe2(Object arg1, Object arg2) {  
        if (arg1 == null || arg2 == null) {  
            throwNpeArray(arg1, arg2);  
        }  
    }  
    public static void throwNpeArray(Object... args) {  
        if (args != null) {  
            for (Object arg : args) {  
                if (arg == null) {  
                    throw new NullPointerException();  
                }  
            }  
        }  
    }  
}
```

## 02\_Kotlin data class基础函数优化

data class 文件：

data class Son(val name: String, val teacher: String, var age: Int = 0)

优化点：hashCode/equals/toString方法重构

```
class Son {
    public String name;
    public String teacher;
    public int age;

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (o == null || getClass() != o.getClass()) return false;
        Son son = (Son) o;
        if (age != son.age) return false;
        if (!name.equals(son.name)) return false;
        return teacher.equals(son.teacher);
    }

    @Override
    public int hashCode() {
        int result = name.hashCode();
        result = 31 * result + teacher.hashCode();
        result = 31 * result + age;
        return result;
    }

    @Override
    public String toString() {
        return "Son{" +
            "name='" + name + '\'' +
            ", teacher='" + teacher + '\'' +
            ", age=" + age +
            '}';
    }
}
```



```
public abstract class AbstractFather {

    public abstract Object[] getObjects();

    public boolean equals(Object other) {
        if (this == other) {return true;}
        if (other == null) {return false;}
        if (!(this.getClass().isAssignableFrom(other.getClass()))) {
            return false;
        }
        Object[] origins = (this).getObjects();
        Object[] others = ((AbstractFather) other).getObjects();
        for (int i = 0; i < origins.length; i++) {
            if (!Intrinsics.areEqual(origins[i], others[i])) {
                return false;
            }
        }
        return true;
    }

    public int hashCode() { return Objects.hash(getObjects()); }

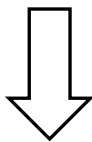
    public String toString() {
        Object[] objects = getObjects();
        int length = objects.length;
        StringBuilder sb = new StringBuilder(this.getClass().getName());
        for (int i = 0; i < length; i++) {
            if (i == 0) {
                sb.append(":%s");
            } else {
                sb.append(",%s");
            }
        }
        return new Formatter().format(sb.toString(), objects).toString(); // "Son2:%s,%s,%s"
    }
}

public class Son2 extends AbstractFather {
    public String name;
    public String teacher;
    public int age;
    @Override
    public Object[] getObjects() { return new Object[]{name, teacher, age}; }
}
```

## 02\_Redex RenamePass

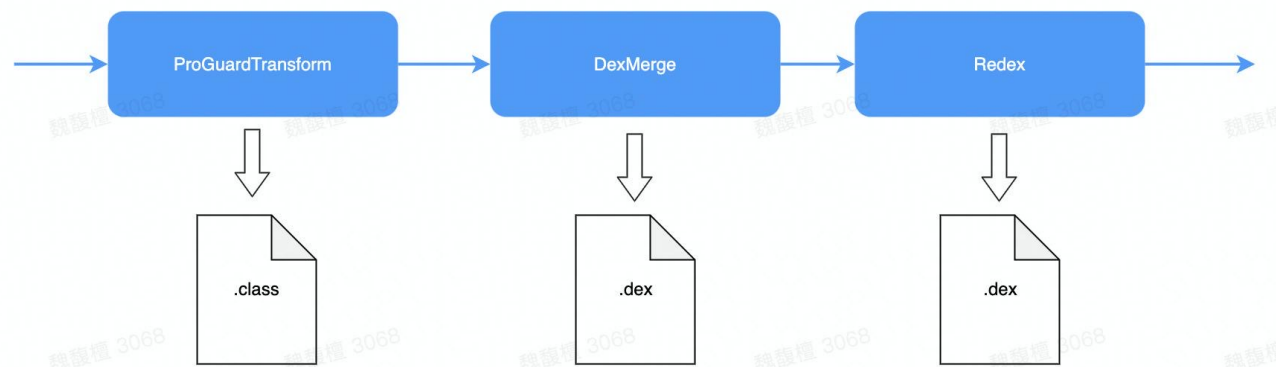
```
androidx.activity.OnBackPressedCallback -> androidx.activity.c:  
    boolean mEnabled -> a  
    java.util.concurrent.CopyOnWriteArrayList mCancellables -> b  
46:56:void <init>(boolean) -> <init>  
91:94:void remove() -> a
```

缩短被混淆类的包名长度



```
androidx.activity.OnBackPressedCallback -> X.c:  
    boolean mEnabled -> a  
    java.util.concurrent.CopyOnWriteArrayList mCancellables -> b  
46:56:void <init>(boolean) -> <init>  
91:94:void remove() -> a
```

为什么不用ProGuard repackaging?



ProGuard在Dex生成前缩短包名，会改变Dex中的Class集合导致优化**结果不稳定**。

Redex在Dex生成后缩短包名，不会改变Dex中的Class集合，优化**结果更稳定**。

# 02\_资源优化

|        | 删                                                               | 缩                                        | 挪               |
|--------|-----------------------------------------------------------------|------------------------------------------|-----------------|
| 图片/xml | <u>shrinkResource</u> 优化    相似图片合并<br><br><u>layout</u> 无用字符串去除 | <u>图片webp/pngquant</u> 优化 <u>View</u> 混淆 | <u>CDN</u> 动态下发 |
| arsc   | <u>dpi</u> 缩减 <u>config</u> 缩减 <u>md5</u> 去重                    | <u>资源路径</u> 缩短 <u>arsc</u> 压缩            | 高dpi插件下发        |
| 音视频/字体 | <u>无用Assest</u> 分析                                              | <u>码率</u> 降低 <u>font</u> 裁剪              | <u>CDN</u> 动态下发 |

- 图片编码组合优化，编译期自动替换
- ARSC文件格式和内容的极致压缩
- XML文件格式深度优化

## 03\_layout/xml 无用字符串去除

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:orientation="vertical"
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <TextView
        android:textSize="20dp"
        android:id="@+id/text_id"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="hello world!" />

</LinearLayout>
```

Layout源文件



```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:orientation="1"
    android:layout_width="-1"
    android:layout_height="-1">

    <TextView
        android:textSize="dimension(5121)"
        android:id="@ref/0x7f08009f"
        android:layout_width="-1"
        android:layout_height="-2"
        android:text="hello world!" />

</LinearLayout>
```

优化前二进制文件



系统会根据属性的int来读取属性的值，其字符串名可以删掉  
frameworks/base/core/jni/android\_util\_AssetManager.cpp

```
// 通过属性整型id获取属性值
static jboolean android_content_AssetManager_applyStyle(...) {
...
    while (ix < NX && curIdent > curXmlAttr) {
        ix++;
        curXmlAttr = xmlParser->getAttributeNameResID(ix); //获取属性id
    }
    if (ix < NX && curIdent == curXmlAttr) { //通过id来标识属性
        block = kXmlBlock;
        xmlParser->getAttributeValue(ix, &value); //获取属性值
        ...
    }
    ...
}
```

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
    xmlns:=""
    ="1"
    ="-1"
    ="-1">

    <TextView
        ="dimension(5121)"
        ="@ref/0x7f08009f"
        ="-1"
        ="-2"
        ="hello world!" />

</LinearLayout>
```

收益**2.6MB**

优化后二进制文件



# 02\_ So库优化

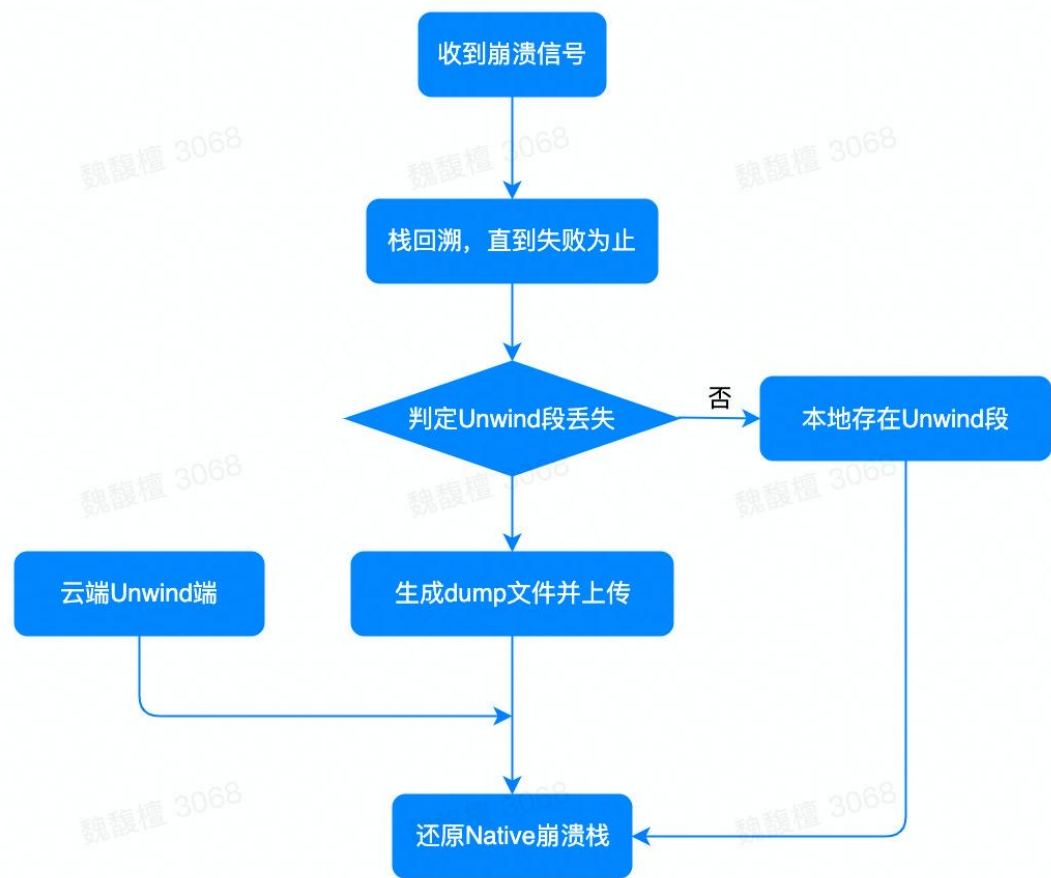
|    | 删                                                                                                                                                       | 缩                         | 挪                          |
|----|---------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------|----------------------------|
| SO | <div><div><u>c++ static</u>动态链接</div><div>导出符号收敛</div><div>无用函数裁剪</div><div><u>-Oz</u>优化</div><div>RTTI优化</div><div><u>c++ exception</u> 移除</div></div> | <div><u>icf</u>代码折叠</div> | <div><u>Unwind</u>优化</div> |

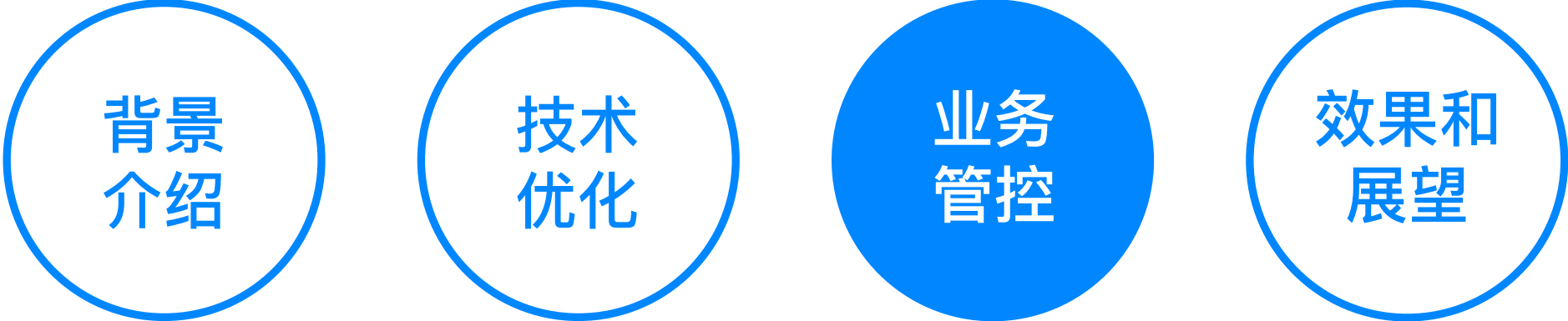
- 针对SO编译参数的优化
- SO静态链接库的扫描和合并
- 基于ELF文件格式的压缩

## 02\_ So库优化-unwind优化

- 将So库unwind段挪到云端,
- 在App端收集信息并上传到云端,
- 包体积收益5-20%, So文件越大收益占比越高。

| FILE SIZE |        | VM SIZE |        |                |
|-----------|--------|---------|--------|----------------|
| 62.3%     | 56.6Ki | 56.5%   | 56.5Ki | .text          |
| 0.0%      | 0      | 11.5%   | 11.5Ki | .bss           |
| 9.4%      | 8.55Ki | 8.5%    | 8.49Ki | .eh_frame      |
| 8.2%      | 7.44Ki | 7.4%    | 7.38Ki | .rodata        |
| 2.9%      | 2.64Ki | 2.6%    | 2.58Ki | .dynsym        |
| 2.8%      | 2.52Ki | 2.5%    | 2.46Ki | .rela.plt      |
| 2.1%      | 1.90Ki | 1.8%    | 1.84Ki | .eh_frame_hdr  |
| 2.1%      | 1.89Ki | 1.8%    | 1.83Ki | .rela.dyn      |
| 1.9%      | 1.73Ki | 1.7%    | 1.67Ki | .plt           |
| 1.3%      | 1.18Ki | 1.1%    | 1.12Ki | .dynstr        |
| 1.0%      | 976    | 0.9%    | 912    | .data          |
| 1.0%      | 976    | 0.9%    | 912    | .got           |
| 1.0%      | 900    | 0.8%    | 836    | .hash          |
| 0.8%      | 752    | 0.3%    | 304    | [6 Others]     |
| 0.7%      | 648    | 0.6%    | 584    | .data.rel.ro   |
| 0.6%      | 592    | 0.5%    | 528    | .dynamic       |
| 0.6%      | 536    | 0.5%    | 536    | [LOAD #0 [RX]] |
| 0.3%      | 285    | 0.0%    | 0      | .shstrtab      |
| 0.3%      | 284    | 0.2%    | 220    | .gnu.version   |
| 0.3%      | 283    | 0.0%    | 0      | .comment       |
| 0.3%      | 272    | 0.0%    | 0      | [Unmapped]     |
| 100.0%    | 90.8Ki | 100.0%  | 100Ki  | TOTAL          |





背景  
介绍

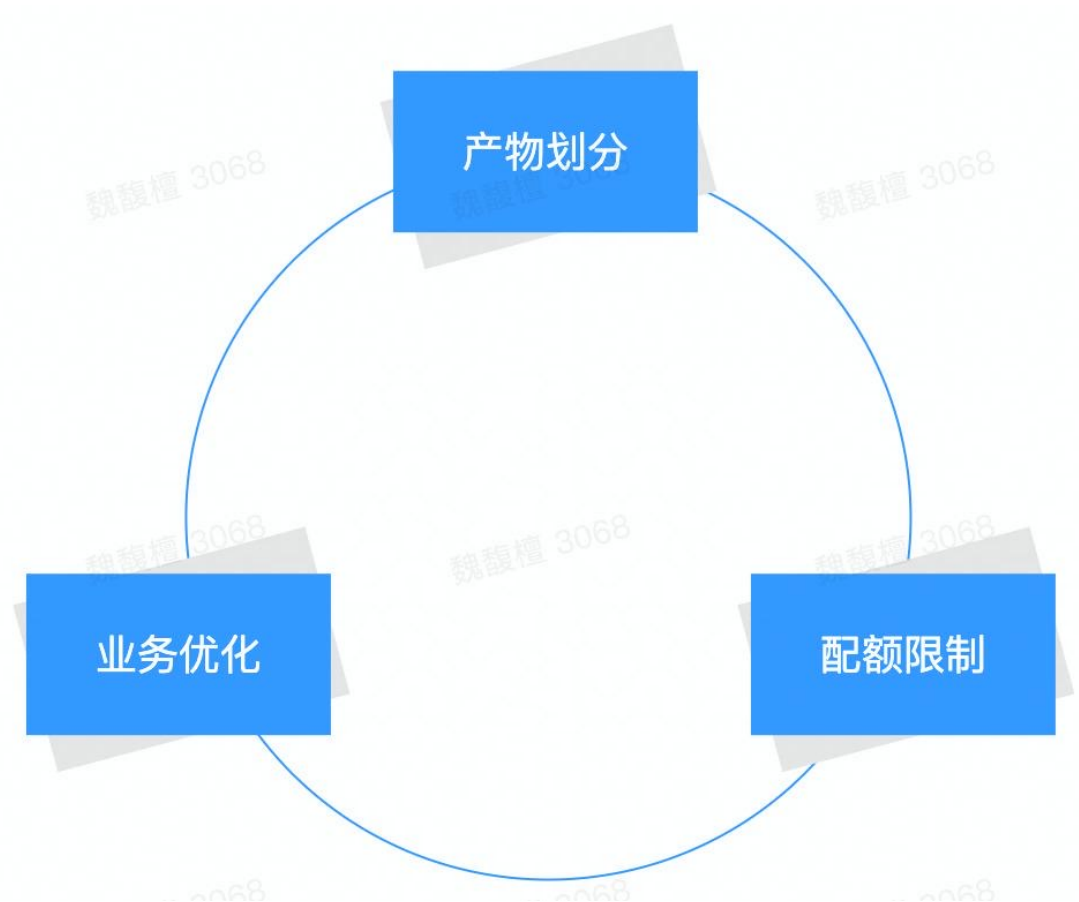
技术  
优化

业务  
管控

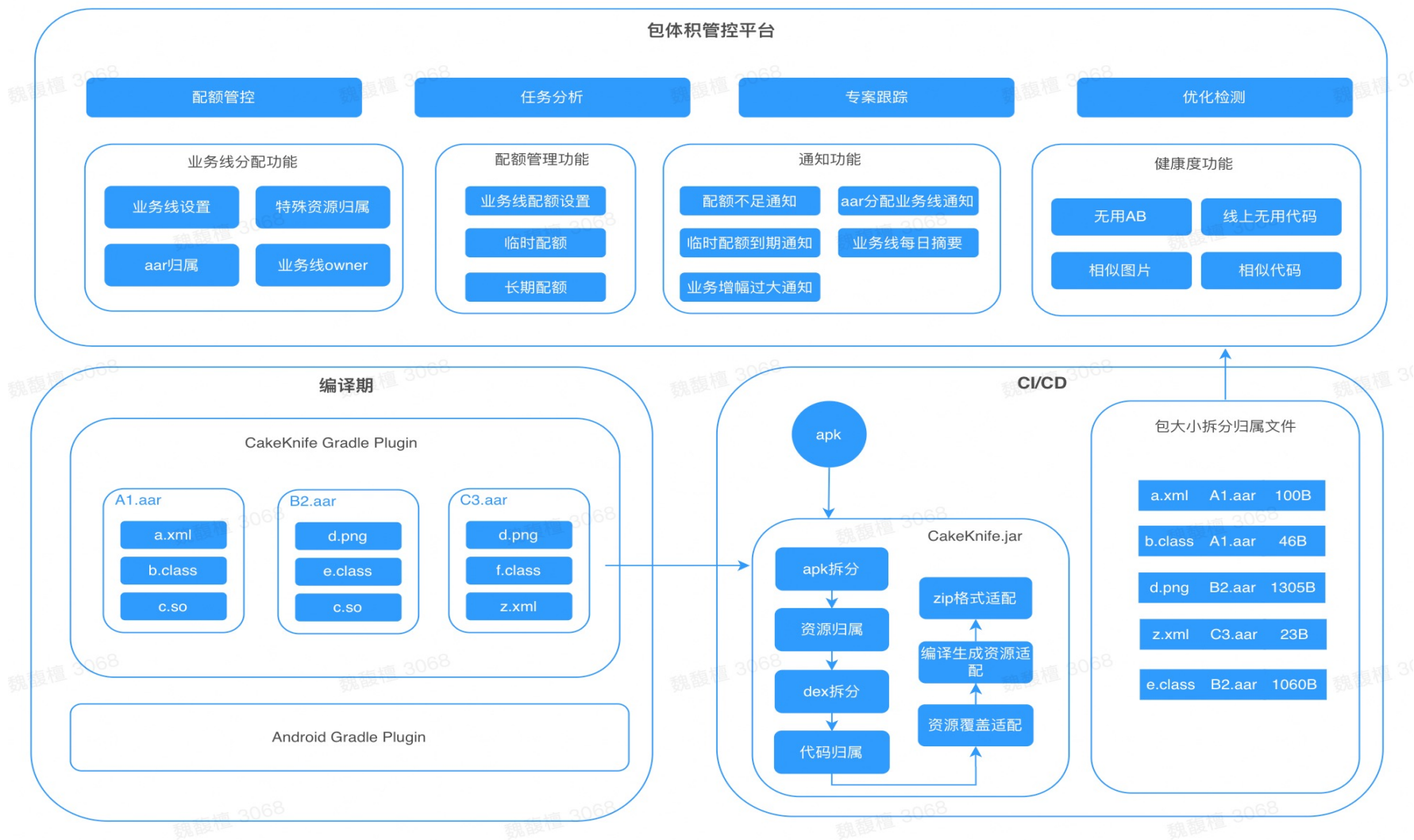
效果和  
展望

## 03\_ 管控目标

对产物大小进行业务线划分，通过配额限制促进业务优化，实现包大小的动态平衡。



# 03\_业务管控系统



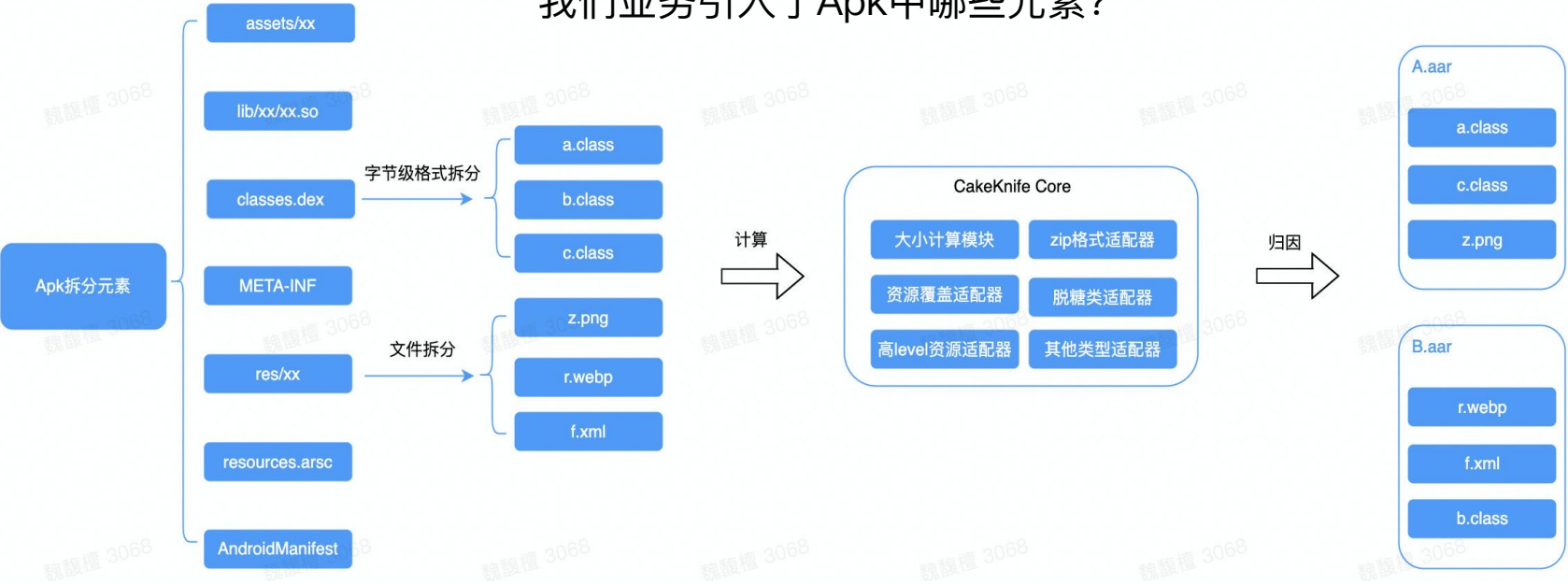
# 03\_ 管控任务示例

- 包大小变化归因到业务线
- 业务线维度配额分配
- 业务线维度限制和管控

| 业务线                                                                                   | 当前大小 ↕                             | 配额 ↕       | 剩余大小      | 符合配额规定 |
|---------------------------------------------------------------------------------------|------------------------------------|------------|-----------|--------|
| >    | 1.77 MiB => 1.76 MiB ( -10.1 kiB ) | 1.78 MiB ⓘ | 12.7 kiB  | ✓      |
| >    | 75 kiB => 74.9 kiB ( -83 B )       | 614 kiB ⓘ  | 539 kiB   | ✓      |
| >    | 3.21 MiB => 3.22 MiB ( +11.1 kiB ) | 3.15 MiB ⓘ | -77.1 kiB | ✗      |
| >    | 1.68 MiB => 1.68 MiB ( -4.09 kiB ) | 1.75 MiB ⓘ | 75.1 kiB  | ✓      |
| >    | 1.86 MiB => 1.85 MiB ( -2.92 kiB ) | 2.02 MiB ⓘ | 171 kiB   | ✓      |
| >    | 166 kiB => 163 kiB ( -2.91 kiB )   | 167 kiB ⓘ  | 3.77 kiB  | ✓      |
| >    | 4.17 MiB => 4.16 MiB ( -8.87 kiB ) | 4.42 MiB ⓘ | 270 kiB   | ✓      |
| >   | 7.56 MiB => 7.54 MiB ( -16.6 kiB ) | 7.6 MiB ⓘ  | 60.9 kiB  | ✓      |
| >  | 4.56 MiB => 4.56 MiB ( -1.64 kiB ) | 4.67 MiB ⓘ | 112 kiB   | ✓      |

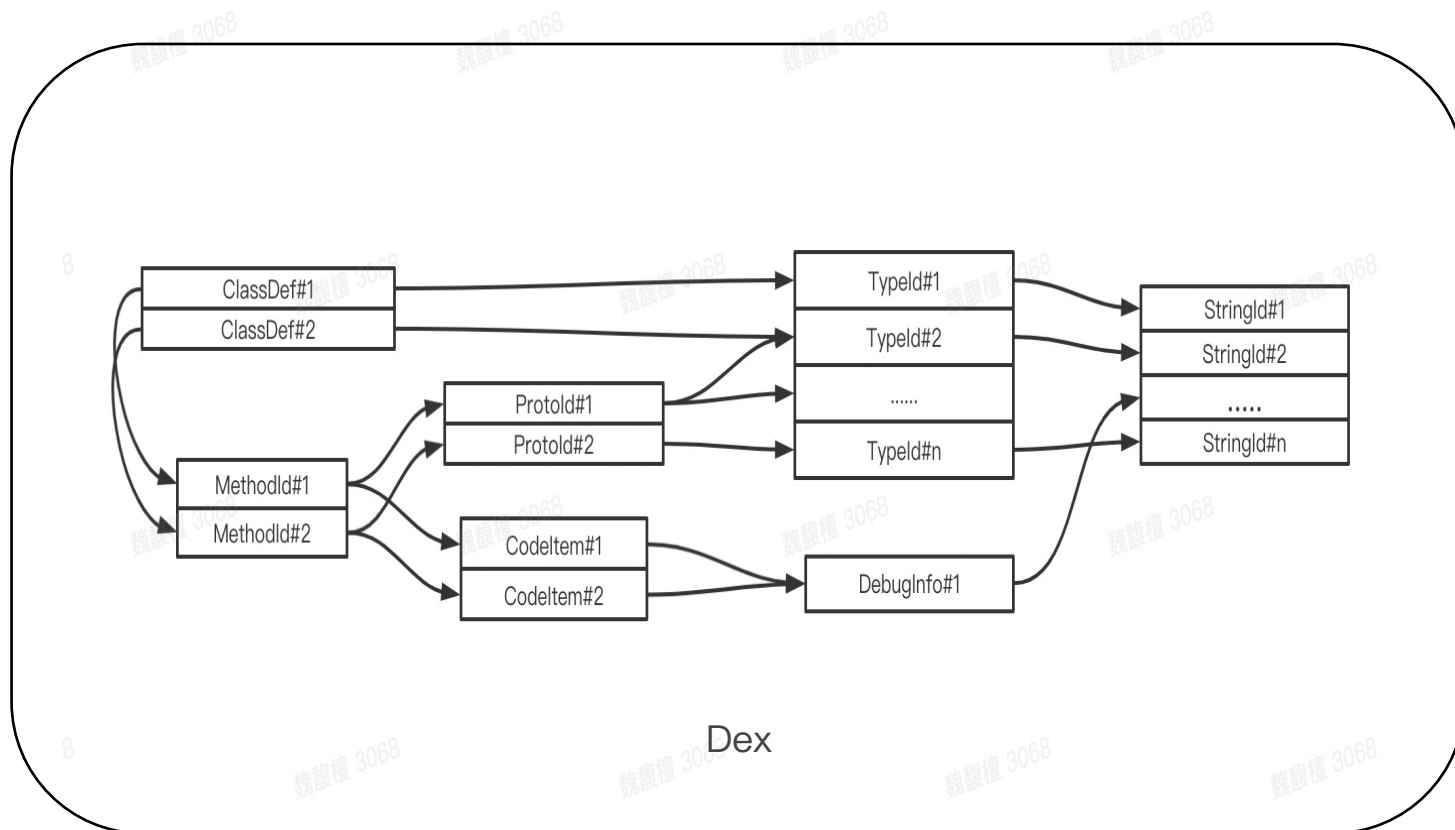
# 03\_业务拆分核心要求

我们业务的大小到底有多大？  
我们业务引入了Apk中哪些元素？



- **精**：小到一个Class，大到一个AAR，都能进行大小归因
- **准**：适配同名资源覆盖、Lambda表达式脱糖等多种编译期修改归因
- **快**：秒级完全归因，更适合CI流程

## 03\_业务拆分代码归因



对Dex中的Class存储格式进行拆分，包含Class自身定义和代码引用的二进制段。

Class#1计算大小:

$\text{MethodId\#1} + \text{TypeId\#1} + \text{StringId\#1} +$   
 $\text{ProtoId\#1} + 1/2 \text{ TypedId\#2} + \text{StringId\#2} +$   
 $\text{CodeItem\#1} + 1/2 \text{ DebugInfo\#1}$

优势:

- 执行时间快
- 统计粒度达Byte级
- 结果真实反应dex大小
- 无中间转化，执行稳定，适配性强



# 03\_业务健康度

## 相似图片检测

|  |                           |
|--|---------------------------|
|  | 有 21 张图片相似, 约可优化 523 kiB  |
|  | 有 2 张图片相似, 约可优化 513 kiB   |
|  | 有 657 张图片相似, 约可优化 314 kiB |
|  | 有 18 张图片相似, 约可优化 314 kiB  |

## 无用代码检测

|                                          |    |              |                 |                     |
|------------------------------------------|----|--------------|-----------------|---------------------|
| com.bytedance.librarian:librarian:0.2.0  | 20 | 0.00% (0/20) | 100.00% (20/20) | 100.00% (6764/6764) |
| com.bytedance.rhea:trace:2.1.2-alpha.86  | 22 | 0.00% (0/22) | 100.00% (22/22) | 100.00% (3831/3831) |
| com.bytedance.rhea:atrace:2.1.2-alpha.86 | 7  | 14.29% (1/7) | 85.71% (6/6)    | 94.34% (1349/1430)  |

|                                                                                                         |                            |           |              |
|---------------------------------------------------------------------------------------------------------|----------------------------|-----------|--------------|
| com.bytedance.android.livesdk                                                                           | liveinteract-impl-cnDouyin | 20.8.0.40 | 中共发现 747 处重复 |
| com/bytedance/android/live/liveinteract/voicechat/wm/LinkInRoomAudioGuestAdapter.java 和其他 1 处有 204 行重复  |                            |           |              |
| com/bytedance/android/live/liveinteract/videotalk/fight/VideoTeamFightViewManager.kt 和其他 1 处有 183 行重复   |                            |           |              |
| com/bytedance/android/live/liveinteract/videotalk/adapters/BaseEqualTalkRoomAdapter.kt 和其他 1 处有 167 行重复 |                            |           |              |

## 相似代码检测

| AB Key                   | Key 状态 | 所属库                                                  | 处理人 |
|--------------------------|--------|------------------------------------------------------|-----|
| InputInterceptDurationAB | 可下线    | com.ss.android.ugc.aweme<br>homepage_common-douyinCn |     |
| PendantLazyInit          | 可下线    | com.bytedance.ies.ugc.aweme<br>spec_act-douyinCn     |     |

## 无用AB检测

## 03\_线上代码覆盖率

原理：在所有Class的静态方法块中  
插桩，在类被加载时将对应Index的  
01状态记录到Bitmap中

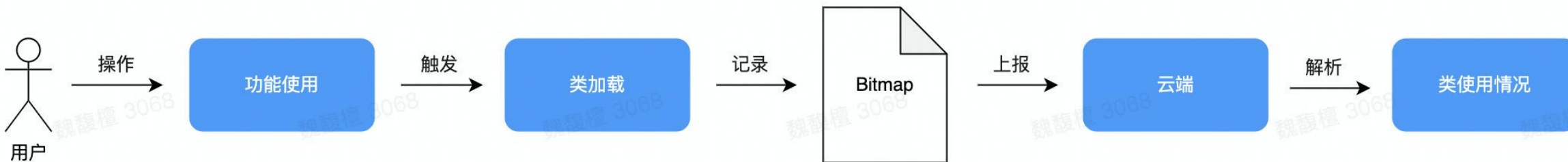
|        |   |   |   |   |   |   |   |   |   |
|--------|---|---|---|---|---|---|---|---|---|
| Index  | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 |
| Bitmap | 1 | 0 | 0 | 1 | 0 | 0 | 0 | 1 | 0 |

0/1状态表示对应类的使用情况

```
public class DouyinPlayer {  
  
    public Player mPlayer;  
  
    public void start() {  
        mPlayer.start();  
    }  
}
```

插桩

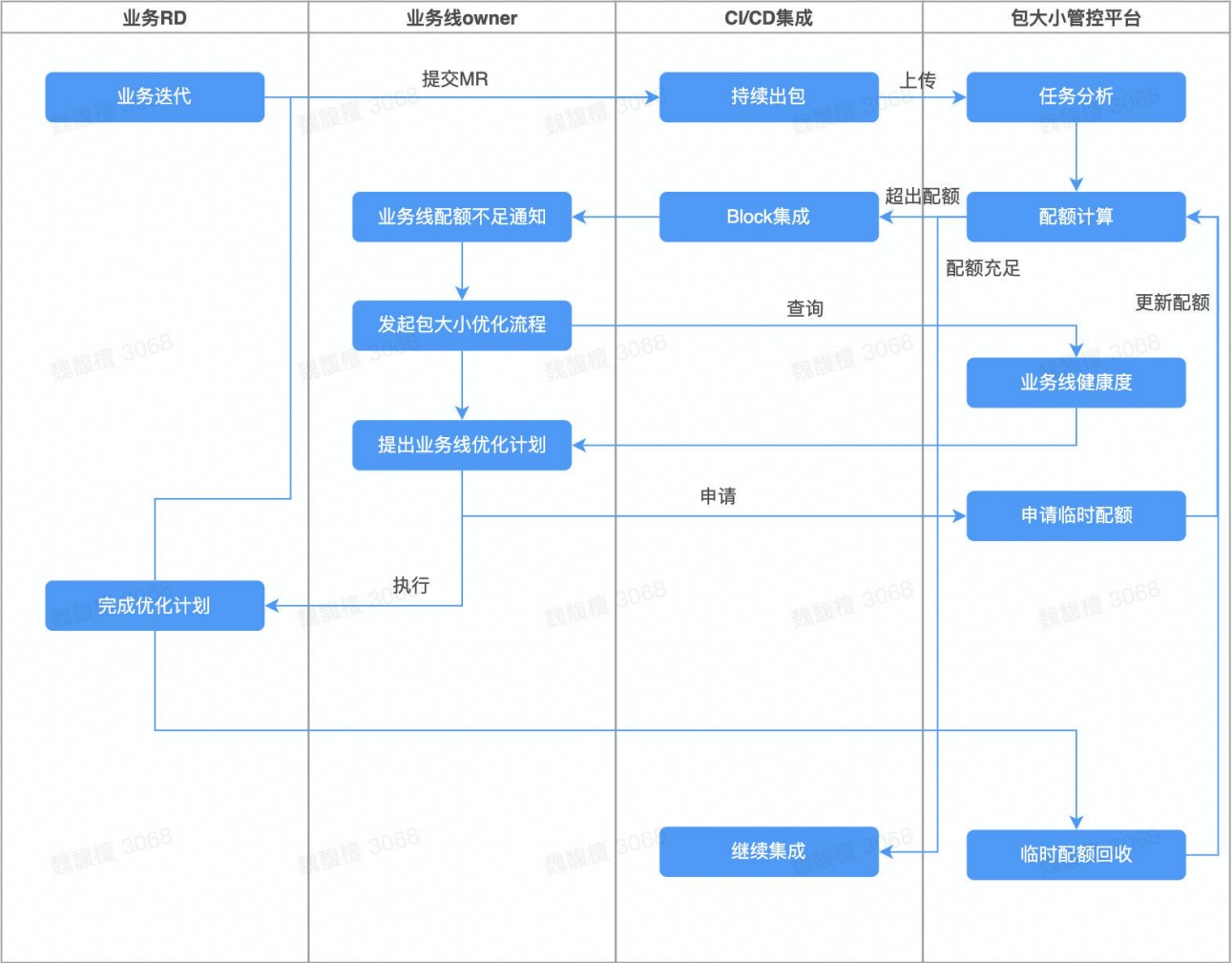
```
public class DouyinPlayer {  
  
    static {  
        Covode.collect(1002);  
    }  
  
    public Player mPlayer;  
  
    public void start() {  
        mPlayer.start();  
    }  
}
```



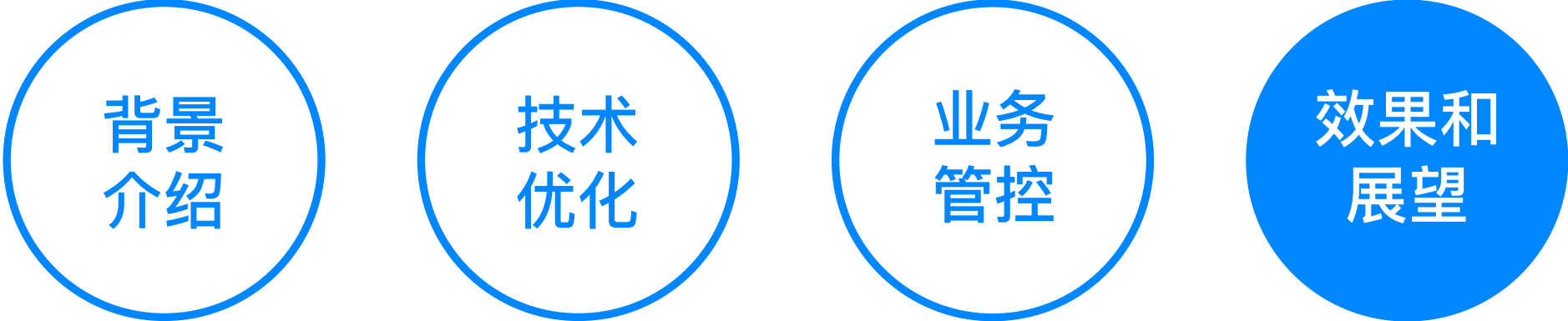
# 03\_线上代码覆盖率



# 03\_业务管控流程



- 业务阈值限制业务合码
- 健康度指标指导业务优化
- 临时配额给予调整时间
- 业务内闭环，去中心化



背景  
介绍

技术  
优化

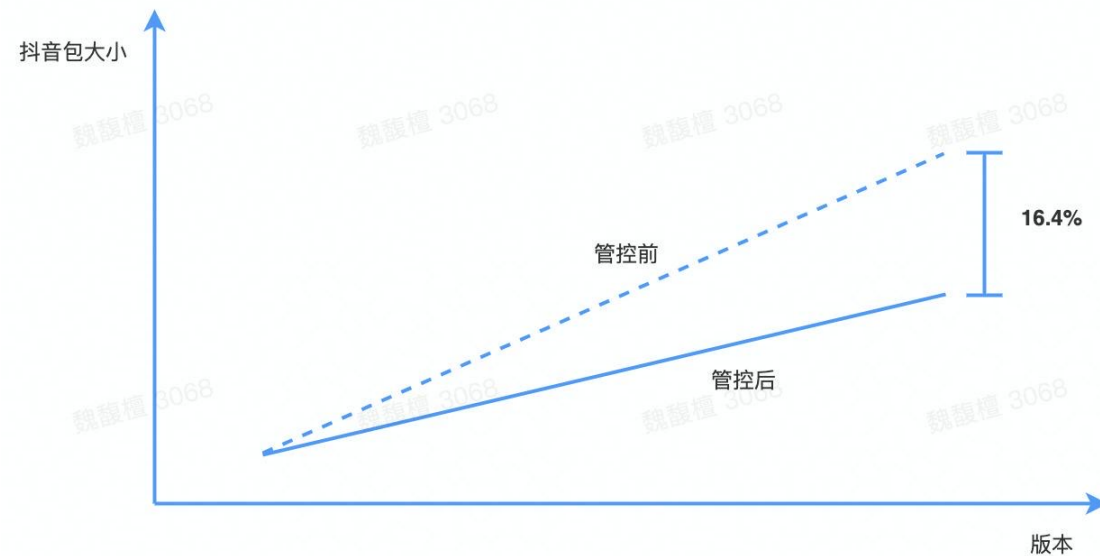
业务  
管控

效果和  
展望

## 04\_效果

100M+

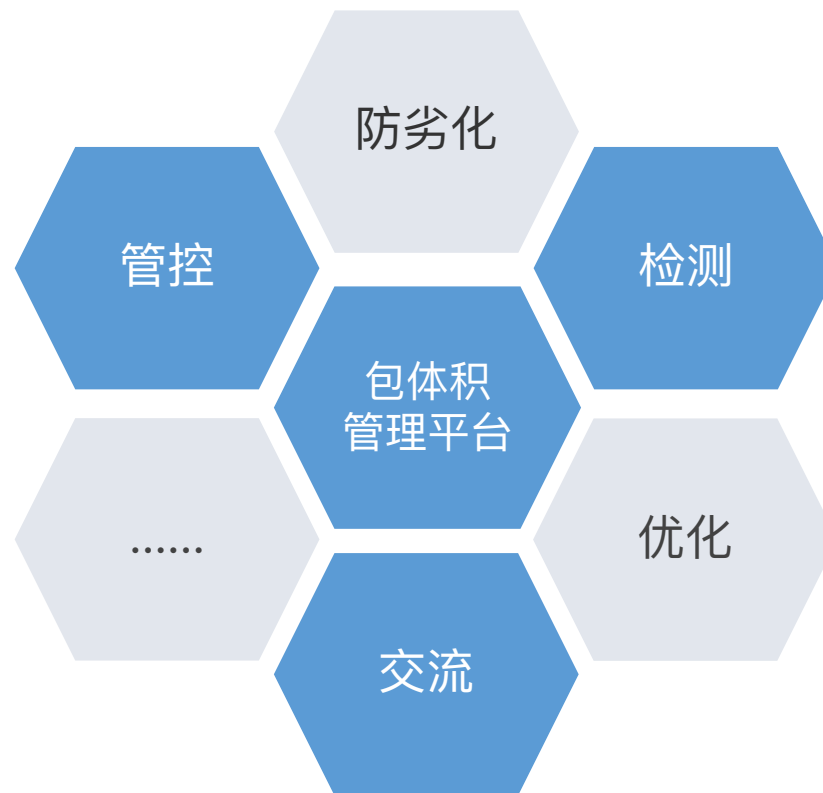
抖音技术优化累计超**100M**



抖音近半年包体积增速下降**16.4%**

## 04\_未来展望

一站式包体积管理平台....



## 04\_ 致谢

感谢张祖桥同学对本次演讲内容的大力支持





**THANKS.**